

Introduction to Collections

Prof. Ashwani Mishra

Assistant Professor
AIML Department

Why Use the Collections Framework?

- The Collections Framework provides reusable data structures that replace hand-written dynamic arrays.
- Standard interfaces such as List, Set, and Map allow code to work with multiple implementations.
- Built-in algorithms for sorting, searching, and copying reduce custom utility code in projects.
- Generic type parameters enforce compile-time type safety and eliminate unsafe casting at runtime.
- Enterprise Java applications rely on collections for caching, configuration, and data transfer objects.

Figure 6.1 Collections Framework Hierarchy

Difference between Array and Collection

	Array	Collection
1	Arrays are fixed in size and hence once we created an array we are not allowed to increase or decrease the size based on our requirement.	Collections are grow-able in nature and hence based on our requirement we can increase or decrease the size.
2	Arrays can hold both primitives as well as objects.	Collections can hold only objects but not primitive.
3	Performance point of view arrays faster than collection	Performance point of view collections are slower than array
4	Arrays can hold only homogeneous elements.	Collections can hold both homogeneous and heterogeneous elements.
5	Memory point of view arrays are not recommended to use.	Memory point of view collections are recommended to use.
6	For any requirement, there is no ready method available.	For every requirement ready made method support is available.

Note: Above difference is same for difference between Array and any Collection object (like Vector, ArrayList, LinkedList).

Figure 6.2 Arrays vs Collections Comparison

Core Collection Interfaces

Core Collection Interfaces

Interface	Description	Common Implementation
List	Ordered sequence, duplicates allowed	ArrayList, LinkedList
Set	Unique elements, no guaranteed order	HashSet, TreeSet
Queue	FIFO processing order	LinkedList, PriorityQueue
Map	Key-value associations	HashMap, TreeMap
Iterator	Traverses elements one at a time	Returned by iterator() method

Figure 6.3 Core Collection Interfaces

Collections Framework Hierarchy



Figure 6.4 Full Interface Hierarchy

ArrayList - Resizable List Implementation

- ArrayList is backed by a dynamic array that grows automatically when capacity is exceeded.
- The add method appends elements to the end, while add(index, element) inserts at a position.
- The get method retrieves an element by zero-based index in constant time on average.
- The remove method deletes the first occurrence of a matching object from the list.
- ArrayList is the default choice when frequent random access by index is required.

Figure 5.5 ArrayList Basic Operations

ArrayListDemo.java

```
import java.util.ArrayList;

ArrayList<String> subjects = new ArrayList<>();
subjects.add("Java");
subjects.add("OOP");
subjects.add("Collections");
subjects.add(1, "Data Structures"); // insert at index 1

System.out.println(subjects.get(0));    // Java
System.out.println(subjects.size());    // 4
subjects.remove("OOP");
System.out.println(subjects.contains("Java")); // true
```

Figure 6.6 ArrayList Basic Operations

List Interface Methods

- The add(E element) method appends a new element to the end of the list and returns true.
- The add(int index, E element) method shifts existing elements to insert at a specific position.
- The get(int index) method returns the element stored at the given zero-based index.
- The remove(Object o) method deletes the first matching element and reports whether removal succeeded.
- The size method returns the current element count, and contains checks membership in the list.

Figure 6.6 Common List Interface Methods

Common List Interface Methods

Method	Return Type	Purpose
add(E element)	boolean	Append element to end of list
add(int index, E e)	void	Insert element at specific index
get(int index)	E	Retrieve element at given index
remove(Object o)	boolean	Remove first matching element
size()	int	Return current number of elements
contains(Object o)	boolean	Check if element exists in list

Figure 6.6 Common List Interface Methods

HashSet - Unique Element Collection

- HashSet stores each element at most once using hash codes for fast lookup and insertion.
- Adding a duplicate value such as 101 twice does not increase the set size beyond unique entries.
- The contains method performs an efficient membership test based on the element hash code.
- Iteration order is not guaranteed and may differ from the order elements were added.
- HashSet is ideal when uniqueness matters more than preserving insertion sequence.

Figure 6.7 HashSet Unique Element Storage

HashSetDemo.java

```
import java.util.HashSet;

HashSet<Integer> rollNumbers = new HashSet<>();
rollNumbers.add(101);
rollNumbers.add(102);
rollNumbers.add(101); // duplicate ignored

System.out.println(rollNumbers.size()); // 2
System.out.println(rollNumbers.contains(102)); // true

for (Integer roll : rollNumbers) {
    System.out.println("Roll: " + roll);
}
```

Figure 6.7 HashSet Unique Element Storage

List vs Set Behavior

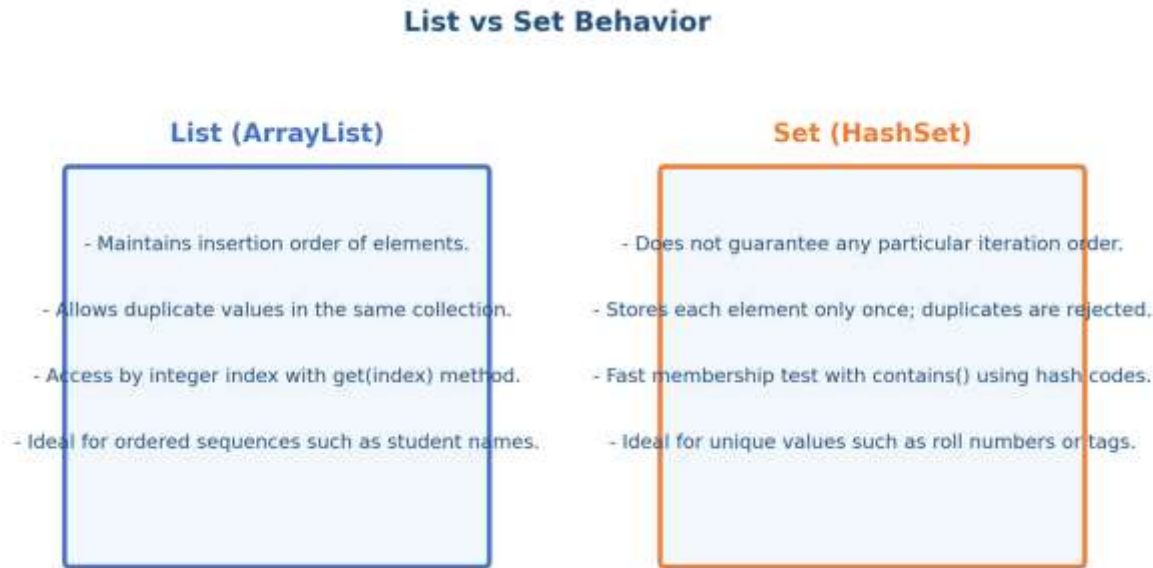


Figure 6.8 List vs Set Comparison

ArrayList vs LinkedList



Figure 6.9 ArrayList vs LinkedList

Iterator and Enhanced for-each Loop

- The Iterator interface provides hasNext and next methods for sequential element traversal.
- Obtaining an iterator via cities.iterator() allows safe removal during iteration with remove().
- The enhanced for-each loop syntax iterates every element without manual index management.
- Internally, the for-each loop compiles to Iterator-based code for Collection implementations.
- Prefer for-each for simple read-only traversal and Iterator when removal during iteration is needed.

Figure 6.10 Iterator and for-each Iteration

```
IteratorDemo.java

import java.util.ArrayList;
import java.util.Iterator;

ArrayList<String> cities = new ArrayList<>();
cities.add("Vadodara");
cities.add("Ahmedabad");
cities.add("Surat");

// Classic Iterator approach
Iterator<String> it = cities.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}

// Enhanced for-each loop (internally uses Iterator)
for (String city : cities) {
    System.out.println(city);
}
```

Figure 6.10 Iterator and for-each Iteration

Map Interface and HashMap

- A Map stores associations between unique keys and corresponding values as key-value pairs.
- HashMap is the most common Map implementation providing average $O(1)$ get and put performance.
- Each key appears at most once; putting a new value for an existing key replaces the old value.
- The `keySet` method returns all keys, enabling iteration over entries in application logic.
- Maps model dictionaries, caches, configuration tables, and lookup structures in real applications.

Figure 5.11 HashMap Key-Value Structure

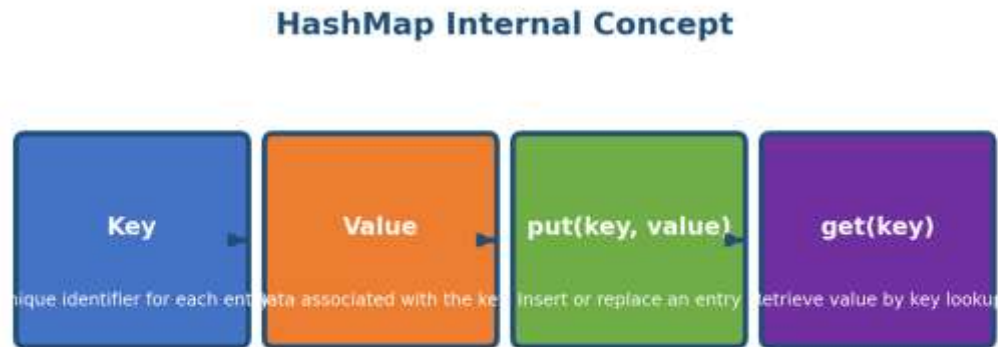


Figure 6.11 HashMap Key-Value Structure

HashMap Code Example

- The put method associates a subject name key with an integer marks value in the map.
- The get method retrieves the value for a given key, returning null if the key is not present.
- The containsKey method checks whether a specific key exists before attempting a lookup.
- Iterating keySet and calling get for each key prints every subject-score pair in the map.
- HashMap is unordered; use LinkedHashMap for insertion order or TreeMap for sorted keys.

Figure 6.12 HashMap put, get, and Iteration

HashMapDemo.java

```
import java.util.HashMap;

HashMap<String, Integer> marks = new HashMap<>();
marks.put("Java", 85);
marks.put("OOP", 92);
marks.put("Collections", 88);

System.out.println(marks.get("OOP")); // 92
System.out.println(marks.containsKey("Java")); // true

for (String subject : marks.keySet()) {
    System.out.println(subject + " -> " + marks.get(subject));
}
```

Figure 6.12 HashMap put, get, and Iteration

Iterating Maps with entrySet

- The entrySet method returns a Set of Map.Entry objects representing each key-value pair.
- Each Map.Entry provides getKey and getValue methods for convenient pair-wise access.
- The enhanced for-each loop over entrySet avoids calling get(key) repeatedly during iteration.
- The values method returns a Collection of all values when keys are not needed in the loop.
- Choosing the right iteration style improves readability and can reduce redundant hash lookups.

Figure 6.13 Map Iteration with entrySet and values

MapIteration.java

```
import java.util.HashMap;
import java.util.Map;

HashMap<String, Integer> scores = new HashMap<>();
scores.put("Java", 90);
scores.put("OOP", 88);

// Iterate key-value pairs with entrySet
for (Map.Entry<String, Integer> entry : scores.entrySet()) {
    System.out.println(entry.getKey() + " = " + entry.getValue());
}

// Iterate values only
for (Integer score : scores.values()) {
    System.out.println("Score: " + score);
}
```

Figure 6.13 Map Iteration with entrySet and values

Introduction to Generics

Raw Types vs Generics

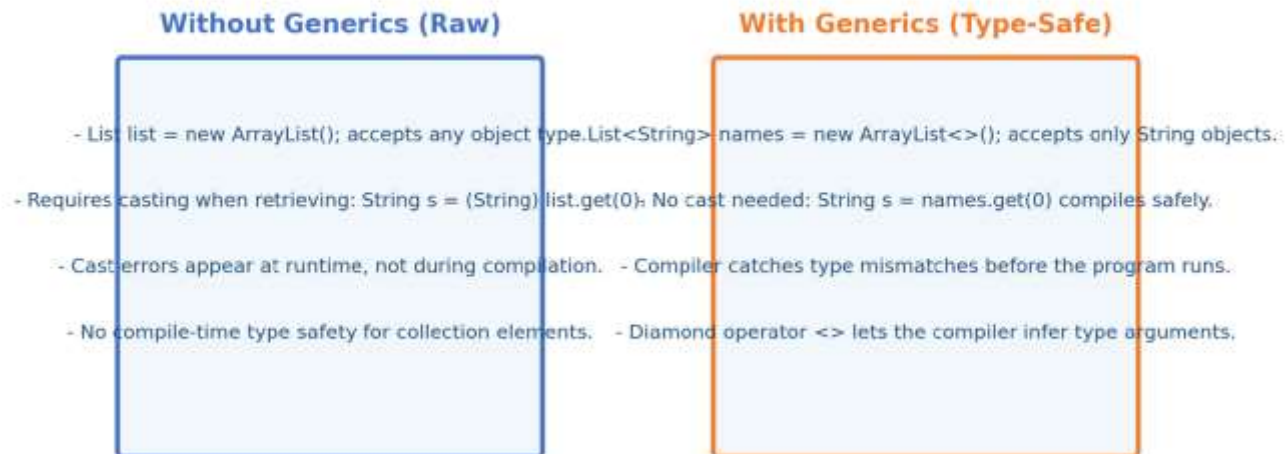


Figure 6.14 Raw Types vs Generic Types

Generic List<String> Example

- Declaring List<String> restricts the list to hold only String objects at compile time.
- Adding an Integer to a List<String> produces a compile error, catching bugs before execution.
- Retrieving elements with get no longer requires an explicit cast to String at runtime.
- The diamond operator in new ArrayList<>() lets the compiler infer the String type argument.
- Generic methods such as printAll<T> work with any element type while preserving type safety.

Figure 6.15 Generic Collections and Methods

GenericsDemo.java

```
import java.util.ArrayList;
import java.util.List;

// Generic List holds only String elements
List<String> names = new ArrayList<>();
names.add("Anit");
names.add("Priya");
// names.add(42); // compile error - Integer not allowed

String first = names.get(0); // no cast required

// Generic method example
public static <T> void printAll(List<T> items) {
    for (T item : items) {
        System.out.println(item);
    }
}
```

Figure 6.15 Generic Collections and Methods

Choosing the Right Collection

- Use ArrayList when you need an ordered sequence with index-based access and possible duplicates.
- Use HashSet when you need fast uniqueness checking and order is not important.
- Use HashMap when you need to look up values quickly by a unique key identifier.
- Apply generic type parameters on declarations to gain compile-time safety and clearer code intent.
- Review time complexity: ArrayList get is $O(1)$, HashSet contains is $O(1)$ average, HashMap get is $O(1)$ average.

Figure 6.16 Choosing the Right Collection Type

Core Collection Interfaces

Interface	Description	Common Implementation
List	Ordered sequence, duplicates allowed	ArrayList, LinkedList
Set	Unique elements, no guaranteed order	HashSet, TreeSet
Queue	FIFO processing order	LinkedList, PriorityQueue
Map	Key-value associations	HashMap, TreeMap
Iterator	Traverses elements one at a time	Returned by iterator() method

Figure 6.16 Choosing the Right Collection Type

Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>

